

Vojtěch Špaček

Links: [GitHub](#) | [LinkedIn](#) | [X/Twitter](#) **Contact:** sparesparrow@protonmail.ch | +420703454180 | [Schedule a call](#)

AI/LLM integration engineer (Model Context Protocol) — agentic tooling for safety-critical embedded & HIL verification | Senior C++ & Python | Linux & networking

sparesparrow - Whoami

- I bring **agentic-AI tooling (Model Context Protocol)** into **safety-critical engineering workflows** — a combination almost nobody else can evidence
- Sole owner of the **HIL verification suite and automated bench** for a safety-certified gas furnace ignition controller (UL/CSA, ANSI Z21.20, NFPA 54) — measured conformance raised to **259 of 271 tracked state transitions (95.6%)**
- Pragmatic **Programmer**, Creative **Problem Solver**
- **Cybersecurity** & **Linux** specialist

Preferred Roles

AI/LLM Developer & Integration Engineer (*Agentic systems, Model Context Protocol, prompt engineering, LLM integration*)

- Built an **agentic AI harness** of specialised agents, skills, prompt workflows, policy hooks and **Model Context Protocol** servers, adopted as a daily part of the Resideo delivery workflow.
- Author of **MCP-Prompts** (97 ★, 21 forks) — published to npm, Docker Hub and the MCP server directories (Glama, MCP Market, PulseMCP), plus a Rust reimplementation on crates.io.
- Advanced prompt engineering: RAG, zero-shot and few-shot, prompt caching, structured outputs, tool calling, encapsulated agents, multimodal routing, and collaborative multi-agent workflows with the human in the loop.

Test Automation / Verification Engineer (*Hardware-in-the-loop, safety-critical verification, instrument control, traceability*)

- **Hardware-in-the-loop bench design and ownership** — instrument control over `pyvisa/USBTC/SCPI` programmable AC sources, `CAN` serial-bus monitoring and `Modbus` relay and probe I/O, with each device owned by a single thread against a shared monotonic clock.
- **State-machine conformance testing** — data-driven generation of ~290 executable tests from a `YAML` catalogue of 270+ transitions, 38 states and 33 events, validated through an eight-gate `Arrange/Act/Assert` pipeline.
- **Safety-certified product verification** (`UL/CSA`, `ANSI Z21.20`, `NFPA 54`) with end-to-end requirement traceability and test-evidence reporting.
- **Unattended nightly pipelines** — per-key locking, crash and abort recovery, result journalling, `Excel` evidence workbooks, bilingual `HTML` dashboards and `Mermaid` failure graphs.
- **Root-cause analysis across instruments** — correlated `JSONL` timelines placing every command, signal edge and bus message on one axis.

AI/LLM Engineering — Approach & Tooling

- Although **C++** is my priority language to work with, I expect a significant programmer's workload transformation in the future, so I have decided to transform myself into **AI Developer**, mastering high-level programming languages (**Python**, **JavaScript**) and advanced prompt engineering.
- Working extensively with all major LLM providers APIs, contributing to open-source frameworks & SDKs, integrating them into traditional tooling and applications, focusing on scalability, modularity, cost optimization, reusability, DRY and automation. Extending traditional tooling and applications, building robust CI/CD pipelines that enable LLM-powered workflows at scale. I excel in natural language defined instructions and system prompts to achieve the best possible results from LLM agents.

Software Developer (**Linux**, **Embedded**, **RTOS**, **TCP/IP**, **Cybersecurity**, **High-availability**, **HAL & IPC**, **Backend**)

- **C++** (*Senior*)
- **Python** (*Senior*)
- **C#** (*Medior*)
- **JavaScript** (*Medior*)
- **Rust**, **Qt/QML** (*working knowledge*)

Relevant Professional Experience - Overview:

- **Resideo (Ademco)** — CoolPeople contractor (01/2026 – 09/2026): Hardware-in-the-loop verification of a safety-certified two-stage gas furnace ignition controller (**UL/CSA**, **ANSI Z21.20**, **NFPA 54**). **Python**, **C#**, **pyvisa/USBTMC/SCPI**, **CAN**, **Modbus**, agentic-AI/**MCP** tooling.
- **Thermo Fisher Scientific** (2023-2024): **C++** server-side software for high-resolution detector hardware operation and image acquisition, **C#** WPF application for image processing and display, **Python** (tests), **HAL/IPC** (**COM/MIDL**), **FPGA** embedded systems, **gRPC**, image acquisition & processing.
- **Honeywell** (2022-2023): **C++** real-time communication APIs, **Python** (tests), **ARM/RTOS** embedded software, aviation standards compliance, Enterprise Architect (UML diagrams).
- **Trusted Network Solutions** (2020-2022): Kernun Adaptive Firewall, **C++**, **Linux**, **Netlink**, **nftables**, **TLS inspection**, **Perl** (tests), **JavaScript** (proxy configuration backend), **VirtualBox**, **GDB**.

Education

- **Masaryk University - Faculty of Informatics**
- Computer Science coursework — *no degree awarded*. Programming, Linux, Networking.
- **Coursera.org**, **DeepLearning.ai**
- **Deep Learning Specialization** (four Machine Learning courses from Andrew Ng), Data Analysis, Prompt Engineering, Building LLM agents.
- **Design Patterns**, **Hack The Box**
- **Software engineering at Google**, **The Pragmatic Programmer**, **Clean Architecture**

Languages & Work Authorization

- **Czech** (native), **Slovak** (native), **English** (C1), **German** (basic)
 - **EU citizen (Czech Republic)** — no work permit required in the EU/EEA. Available as an independent contractor (Czech IČO); Brno on-site and remote.
-

Professional Experience - Detailed

Resideo (Ademco) - CoolPeople Contractor

Test Automation Engineer (Python, C#) (01/2026 – 09/2026): Hardware-in-the-loop verification of a safety-certified two-stage gas furnace ignition controller (**UL/CSA**, **ANSI Z21.20**, **NFPA 54**), as sole owner of the verification test suite and of the automated bench that drives it.

- Designed an **Arrange/Act/Assert** validation pipeline in **Python** with eight independent gates covering preconditions, stimulus timing, state sequence, alarms, signal history, cross-layer consistency, timer accuracy and requirement traceability.
- Built a data-driven generator expanding a **YAML** catalogue of 270+ state transitions, 38 states and 33 events into roughly 290 executable tests, removing hand-written duplication.
- Engineered the bench device layer so each instrument is owned by a single thread against a shared monotonic clock, covering **pyvisa/USBTMC/SCPI** programmable AC source, **CAN** serial-bus monitoring and **Modbus** relay and probe I/O.
- Implemented a correlated **JSONL** timeline placing every instrument command, signal edge and bus message on one axis, enabling root-cause analysis across three independent devices.
- Diagnosed and fixed a leaked USB handle that had been misreported as a hardware fault for months, restoring an entire blocked test family.
- Raised measured conformance to 259 of 271 tracked transitions (95.6%) and closed a ~148-case gap in which stimulus windows were never actually measured, bringing it to zero.
- Delivered an unattended nightly bench pipeline with per-key locking, crash and abort recovery, result journalling, **Excel** evidence workbooks, bilingual **HTML** dashboards and **Mermaid** failure graphs, cutting report generation from 15.7 s to 1.6 s.
- Investigated 25+ firmware and harness defects against the **C/C++** firmware sources, shipping a standalone reproduction script for each.
- Migrated a **C#/ .NET WinForms** serial-bus monitor into version control, fixed a log-parser crash and reduced its build and deployment to one command.
- Created an agentic **AI** harness of specialised agents, skills, prompt workflows, policy hooks and **Model Context Protocol** servers, adopted as a daily part of the delivery workflow.
- Owned delivery end to end, including requirement traceability, issue tracking, branch and pull-request workflow, **Conan** packaging, CI integration and monthly client reporting.

New Skills Learned: **Hardware-in-the-Loop Test Design**, **Safety-Certified Product Verification (UL/CSA, ANSI Z21.20, NFPA 54)**, **Ignition State-Machine Domain Knowledge**, **pyvisa / USBTMC / SCPI Instrument Control**, **Modbus I/O Cards**, **CAN Serial-Bus Diagnostics**, **Deterministic Multi-Threaded Device Ownership**, **Test-**

Evidence & Traceability Reporting (openpyxl, HTML dashboards, Mermaid), Model Context Protocol Server Development, Agentic Workflow Design & Governance

Thermo Fisher Scientific - Ixperta Contractor

Software Developer (C++): Electron Microscopy software development for high-resolution detector hardware operation and image acquisition.

- Implemented server-side software in C++, including hardware abstraction layers (HAL) and inter-process communication (IPC) using COM and MIDL on Windows.
- Developed embedded software for FPGA hardware, utilizing network communication protocols (gRPC) in C++, Python, and Linux environments.
- Contributed to WPF application development on the backend using C# and COM/MIDL IPC with HAL, and ensured graceful degradation of the application.
- Developed vscode-automation, an enterprise-grade VSCode extension integrating TICS code quality analysis with OpenAI API for automated violation detection and AI-powered remediation, streamlining code quality workflows for the development team using TypeScript, VSCode Extension API, and Git API integration.
- Led 3 knowledge sharing sessions on GitHub Copilot best practices for the development team and presented bi-weekly stakeholder demos showcasing sprint deliverables via GitHub Teams, including final presentation of vscode-automation innovation sprint outcomes.

New Skills Learned: Image Acquisition & Processing, C#, .NET, Component Object Model, Software Development for Windows, gRPC, FPGA development, VSCode Extension Development, TypeScript, OpenAI API Integration, Technical Presentation & Stakeholder Communication

Honeywell

- **Project:** Onboard Maintenance System that monitors the condition of aircraft and diagnoses issues during flight.
- Architected and implemented DiagTest framework, a sophisticated state machine-based diagnostic test engine in C++ managing 15+ operational states for aircraft member system testing during flight and maintenance operations.
- Designed job-based execution system with DiagTestJob and DiagTestMsgQueProc for asynchronous test orchestration, implementing message queue processing patterns for reliable real-time diagnostics.
- Engineered test interference management system preventing conflicting diagnostic tests from running simultaneously, ensuring aircraft system safety and data integrity.
- Implemented four diagnostic test modes (data collection, passive monitoring, interactive, non-interactive) with timeout management, precondition handling, and aviation-standard command/response protocols.
- Developed comprehensive integration tests using Python to validate communication protocols, state transitions, and system stability under various failure scenarios.
- Built real-time parameter monitoring system (MonitorManager) tracking aircraft system values during test execution with maintenance message correlation.

New Skills Learned: Embedded Software, ARM, RTOS, Clang, Cross-platform Compilation, Flatbuffers (binary data serialization, C++ headers generation), Conan (Dependency management), Enterprise Architect (UML diagrams), Aviation Safety Standards, State Machine Design Patterns, TeamCity, GitLab CI/CD

Trusted Network Solutions

- **Implemented a forward proxy** using C++ on Linux, featuring HTTP/HTTPS proxy with TLS/SSL inspection, OCSP integration for certificate validation, Kerberos authentication, and LDAP integration for user/group lookup.
- Built comprehensive proxy infrastructure with certificate generation and management (CA, client certificates), dynamic configuration builder/parser system, and manager/partition architecture for modular daemon design.
- Integrated with security components including Suricata (Intrusion Detection System) and nftables (Static Firewall), aggregating logs and managing connection information in PostgreSQL.
- Developed DNS resolution and caching mechanisms, domain categorization, and client authentication methods.
- Developed and maintained ipmon, a sophisticated network monitoring solution demonstrating deep expertise in:
- Implemented real-time network interface monitoring using NETLINK sockets, handling both IPv4 and IPv6 address management.
- Engineered robust socket programming with Unix domain sockets for inter-process communication.
- Designed and implemented integration with nftables for dynamic firewall rule management.
- Created JSON-based configuration management for network settings and proxy configurations.
- Implemented thread-safe networking operations using modern C++ features including mutex locks and RAII patterns.
- Designed event-driven architecture for real-time network changes with configurable delay mechanisms.
- Created atomic file operations for configuration management ensuring system stability.
- Developed reporter_log C++ replacement of Perl command, implementing PostgreSQL database integration, structured log parsing and processing, and Perl integration for legacy compatibility with database schema management.
- Built configuration generation system using TypeScript/JavaScript, implementing YAML-based high-level configuration processing, certificate generation automation, and integration with web backend for dynamic proxy configuration.
- Developed comprehensive testing infrastructure with Perl test framework, implementing HTTP/HTTPS protocol testing, TLS/SSL inspection testing, Kerberos/LDAP authentication testing, and OCSP validation testing for enterprise-grade quality assurance.

New Skills Learned: TCP Proxy, HTTP Proxy, Encrypted Traffic Inspection, TLS 1.3, C++20, C++17, Boost Libraries, JSON-CPP, Kerberos/Heimdal, OCSP, LDAP, SQLite,

Perl, JavaScript/TypeScript, YAML, Jenkins, Linux Networking, Netlink, UNIX Sockets, SSH Tunneling, Port Forwarding

Self-employed

- **B2C:** *IT Support Freelance, Consumer Hardware Retail, Security Consulting.*
- **B2B:** *CI/CD Automation Pipelines, Linux Server Administration* **New Skills Learned:** Kali Linux, Wireshark, TeamCity, Docker. Self-studied for Certified Ethical Hacker, never took the exam.

IBM - Automation Developer / IT Systems Specialist

- Built an automation application that **replaced manual human workload** (ticketing, basic server administration), using **Bash**, **PowerShell** and **JavaScript** and applying **Automata Theory** to design state-driven workflows.
- Monitored **Unix/Windows** production servers and resolved incidents; hands-on with **Active Directory** and **VMware**.

New Skills Learned: Bash, PowerShell, JavaScript, Practical usage of Automata Theory, RedHat Linux, Windows Server, Active Directory, VMware

Open Source Contributions

MCP-Prompts (97 ★ | 21 forks):

- Comprehensive **Model Context Protocol server** for managing prompt templates and LLM interactions.
- **Four core MCP implementations:** filesystem operations, memory management, AWS integration, and generic MCP configurations.
- Features **Docker containerization** across multiple server types, **AWS deployment examples** (S3, Lambda, DynamoDB, CloudWatch, IAM, ECR), and **comprehensive documentation**.
- Supports development workflows, AWS infrastructure, data processing, and system administration through specialized prompt templates.
- *mcp-prompts-rs: Reimplementation in Rust, optimized for memory efficiency and high concurrency while maintaining full MCP protocol compatibility.* **Repositories & Marketplace:**
- [GitHub](#)
- [npm package](#)
- [Docker Image](#)
- **Community impact (MCP Server Hubs & Markets directories):** [Glama.ai](#), [Glama \(Fly.dev\)](#), [MCP Market](#), [MagicSlides](#), [MCPHub](#), [npm](#), [Docker Hub](#), [Skywork](#), [PulseMCP](#), [AIBASE](#)

mcp-project-orchestrator:

- **Template orchestration** with pattern recognition and code generation.
- **JSON-driven orchestration** analyzing user input to identify design patterns and create initial project files from templates.

- **Template management** with project/component templates and variable substitution, plus prompt template rendering.
- **Mermaid diagram generation** (flowcharts, sequence diagrams, class diagrams) with SVG/PNG rendering.

human-action: Audiobook generation pipeline with Elevenlabs Python SDK and voice (Python)

hard-coder:

- Demonstrates **early adoption of agentic design patterns** and multi-step reasoning in AI code generation
- **Reasoning Engine** AI Assistant with implemented **reasoning & planning capability**, designed when LLMs lacked advanced reasoning without agentic frameworks.
- Transforms ideas into working code including **tests and documentation**.
- Evolution path: **OpenAI GPT Store** → **Anthropic** migration → **CrewAI** refactoring → **Cursor IDE Composer Notepad**.

github-events: Python service monitoring 23+ GitHub event types with analytics (Python)

- Comprehensive monitoring service tracking GitHub activities with **repository health scoring**, **developer productivity analysis**, and **security anomaly detection**.
- **FastAPI REST API** with 15+ endpoints, **interactive dashboard** with live data integration, **MCP server** support.
- **Dual backend support:** SQLite and AWS DynamoDB with **ecosystem monitoring** for cross-domain cooperation.

sparetools: monorepo aggregating shared Python tools, Conan recipes, demonstrating C/C++ dependency management and cross-platform CI/CD.

cpy: Bundled CPython for sparetools with Python runtime

rtp-midi: Modular Rust architecture for real-time MIDI, audio, and LED control (Rust 66.5%, 2★)

- **Low-latency MIDI routing** connecting music production hardware to DAWs and addressable LED systems via network protocols.
- **Android Hub** (Kotlin + Rust NDK) with foreground service, AMidi NDK, mDNS discovery, plus **ESP32 visualizer** using FastLED and dual-core FreeRTOS.
- **Dual-protocol architecture:** RTP-MIDI for DAWs, OSC for LED devices with modern CI/CD pipeline and Docker containerization.

Podman-Desktop-Extension-MCP: combining multiple agent containers for collaboration and routing between them.

ipmon: Netlink socket monitoring daemon that detects real-time IPv4/IPv6 address changes, and updates firewall (nftables) rules accordingly. Built with modern C++20, applying patterns such as RAII, move semantics, thread-safe and atomic operations.

Event-driven architecture, Unix domain sockets for IPC, JSON configuration management. Packaged for Debian-based systems with complete systemd hardening profiles for production deployment.

rust-network-mgr: Redesigned and reimplemented ipmon as a Rust daemon orchestrating dynamic firewall rules via Netlink with minimal memory footprint. Architected event-driven system for real-time network state changes, implementing atomic firewall updates to prevent race conditions and inconsistent rulesets during topology changes. Built with lock-free data structures and epoll-based multiplexing for high-throughput event processing. | crates.io/crates/rust-network-mgr